cassandra_report

February 10, 2025

1 Cassandra Report: OCC2 09-earthquakes

- Klink, Carl
- Lefebvre, Romain
- Matthews, Louis-Marie
- Muller, Julie

All the code of this notebook was run in an environment running Python and Cassandra on Linux. In particular, a Docker container was used that was running Cassandra. See the appendices for the `docker-compose.yml` definition.

1.1 Dataset preparation

We are working with the `earthquakes_big.geojson.json` dataset. Since this is a JSON file, we will first explore what it contains with Pandas, and convert it to an appropriate format before uploading it into a Cassandra table.

[1]:
```
JSON_FILENAME = 'earthquakes_big.geojson.json'
```

[2]:
```
import pandas as pd
```

Let's read our file.

[3]:
```
df = pd.read_json(JSON_FILENAME, lines=True)
```

We notice the type column is always equal to "Feature". Indeed:

[4]:
```
df['type'].unique()
```

[4]:
```
array(['Feature'], dtype=object)
```

So we can drop it.

[5]:
```
df.drop('type', axis=1, inplace=True)
```

We notice some types are nested (`properties` and `geometry`). We can flatten the data frame.

[6]:
```
df_properties = pd.json_normalize(df['properties'])
df_geometry = pd.json_normalize(df['geometry'])
```

We notice the `type` column always hold the same value.

[7]:
```
df_geometry['type'].unique()
```

[7]:
```
array(['Point'], dtype=object)
```

So we can drop the `type` column.

[8]:
```
df_geometry.drop('type', axis=1, inplace=True)
```

We can merge all this in a flattened dataframe.

[9]:
```
merged_df = pd.concat([df.drop(['properties', 'geometry'], axis=1),␣
↪df_properties, df_geometry], axis=1)
```

By listing all unique values of each column, we can infer the desired type.

[10]:
```
merged_df['alert'].unique()
```

[10]:
```
array([None, 'green', 'yellow'], dtype=object)
```

[11]:
```
merged_df['code'].unique()
```

[11]:
```
array(['72001620', '72001615', '10729211', …, '71985246', '10709349',
       '2013pucw'], shape=(7669,), dtype=object)
```

And so on and so on. After checking the type of values for all columns, we can apply the desired type.

[12]:
```
merged_df[['alert', 'code', 'detail', 'id', 'magType', 'place', 'net', 'url',␣
↪'status', 'type']] = merged_df[['alert', 'code', 'detail', 'id', 'magType',␣
↪'place', 'net', 'url', 'status', 'type']].astype(pd.StringDtype())
merged_df['time'] = merged_df['time'].astype(pd.Int64Dtype())
merged_df['types'] = merged_df['types'].apply(lambda x: x[1:-1].split(','))
merged_df['sources'] = merged_df['sources'].apply(lambda x: x[1:-1].split(','))
merged_df['ids'] = merged_df['ids'].apply(lambda x: x[1:-1].split(','))
merged_df['updated'] =  (merged_df['updated']/1000).astype('int32')
```

[13]:
```
merged_df.rename(str.lower, axis='columns', inplace=True)
merged_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7669 entries, 0 to 7668
Data columns (total 27 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   id           7669 non-null   string
 1   mag          7668 non-null   float64
 2   place        7669 non-null   string
```

```
 3   time          7668 non-null   Int64
 4   updated       7669 non-null   int32
 5   tz            7669 non-null   int64
 6   url           7669 non-null   string
 7   detail        7669 non-null   string
 8   felt          801 non-null    float64
 9   cdi           801 non-null    float64
 10  mmi           91 non-null     float64
 11  alert         63 non-null     string
 12  status        7668 non-null   string
 13  tsunami       15 non-null     float64
 14  sig           7669 non-null   int64
 15  net           7669 non-null   string
 16  code          7669 non-null   string
 17  ids           7669 non-null   object
 18  sources       7669 non-null   object
 19  types         7669 non-null   object
 20  nst           4116 non-null   float64
 21  dmin          6385 non-null   float64
 22  rms           7630 non-null   float64
 23  gap           6868 non-null   float64
 24  magtype       7668 non-null   string
 25  type          7668 non-null   string
 26  coordinates   7669 non-null   object
dtypes: Int64(1), float64(9), int32(1), int64(2), object(4), string(10)
memory usage: 1.6+ MB
```

We now save our new JSON to the disk.

[ ]:
```
merged_df.to_json("merged_df.json", orient='records', lines=True)
```

1.1.1 Section summary

In this section:

1. we explored the content and the structure of our JSON, dataset,
2. we removed columns that did not yield any data,
3. we flattened the dataset so that there are no nested datasets,
4. we applied the correct types so that the result is correctly encoded into JSON,
5. and finally, we saved the new JSON file into `merged_df.json`.

1.2 Loading the data into Cassandra

Now that we "normalized" our dataset and saved it to `merged_df.json`, we need to create a keyspace and a table, and load our dataset into the created table. We can import the data automatically into our Cassandra table using an automated tool. This tool is called DSBulk.

1.2.1 Creating the keyspace and the table

From our Cassandra server (*i.e.* in our case, a Docker container, see the appendix), we first connect to our Cassandra local cluster by running `cqlsh`.

We then create and connect to the keyspace.

```
CREATE KEYSPACE ks WITH REPLICATION = { 'class': 'SimpleStrategy', 'replication_factor': 1 };

USE ks;
```

We then create a table based on our `merged_df` structure.

From the information above we can derive our `CREATE TABLE` statement!

```
CREATE table earthquakes (
    id text PRIMARY KEY,
    mag double,
    place text,
    time double,
    updated int,
    tz smallint,
    url text,
    detail text,
    felt double,
    cdi double,
    mmi double,
    alert text,
    status text,
    tsunami double,
    sig int,
    net text,
    code text,
    ids list<text>,
    sources list<text>,
    types list<text>,
    nst double,
    dmin double,
    rms double,
    gap double,
    magtype text,
    type text,
    coordinates tuple<double, double, double>
);

-- Since we only have one node, we tell Cassandra not to wait for update acknowledgement.
ALTER TABLE earthquakes WITH GC_GRACE_SECONDS = 0;
```

1.2.2 Installing DSBulk

We'll then load our data into our keyspace. To do that, we will install and use DSBulk in our container.

While it is possible to import the data from JSON into Cassandra manually, doing so is much less efficient. As a result, we decided to use DSBulk, which is optimised for converting large amounts of data from a certain format, such as JSON, and storing it in Cassandra.

For this, we need to do in order:

1. Check we have Java installed.
2. Download DSBulk.
3. Extract it.
4. Add DSBulk to our `PATH`.
5. Reload our bash profile.

The associated commands are:

```
# Check we have Java installed, should return the vresion of Java
echo $JAVA_HOME

# Download and install DSBulk
wget https://github.com/datastax/dsbulk/releases/download/1.11.0/dsbulk-1.11.0.tar.gz
tar -zxvf dsbulk-1.11.0.tar.gz
mv dsbulk-1.11.0/ /opt/
echo 'PATH=/opt/dsbulk-1.11.0/bin/:$PATH' >> .profile
source .profile
```

> **As already specified at the start of the notebook, all the code and commands of this notebook must be run from the Cassandra server, such as a Docker container.**

1.2.3 Loading the data from JSON into Cassandra

And we can load our dataset into Cassandra!

```
dsbulk load -c json -url merged_df.json -k ks -t earthquakes
```

If you get an error, check that the path to `merged_df.json` is correct.

1.3 Queries

For easier readability we will not put the outputs of every queries, only the shorter ones.

1.3.1 Simple queries

1. Select the first 5 rows from the earthquakes table

```
cqlsh:ks> SELECT * FROM earthquakes LIMIT 5;
ak10713314 |  null | null | 10713314 | (-151.5354, 63.3891, 21.4) | http://earthquake.usgs.gov/
 ci15344617 |  null |    1 | 15344617 | (-117.2918, 34.0795, 16.6) | http://earthquake.usgs.gov
 ak10726053 |  null | null | 10726053 | (-150.2491, 61.4812, 46.6) | http://earthquake.usgs.gov
```

```
 ak10721319 |  null | null | 10721319 |  (-145.2824, 65.6752, 7.6) | http://earthquake.usgs.gov
 ci15344105 |  null | null | 15344105 |  (-115.6078, 33.1712, 1.1) | http://earthquake.usgs.gov
```

2. Select only the ‘place’ and ‘mag’ columns from the earthquakes table, limiting to the first 5 rows

```
SELECT place, mag FROM earthquakes LIMIT 5;
```

place	mag
129km W of Cantwell, Alaska	0.9
2km ENE of Colton, California	2.3
16km WSW of Big Lake, Alaska	1.2
35km NW of Circle Hot Springs Station, Alaska	1.3
10km WNW of Calipatria, California	0.8

3. Select a specific earthquake by its ‘id’ (efficient query since ‘id’ is the primary key)

```
SELECT * FROM earthquakes WHERE id = 'usb000hc3b';
```

id	alert cdi code coordinates detail	dmin felt gap ids mag magtype mmi net nst	place	rms sig sources status time tsunami type types	tz updated url
usb000hc3b	null null b000hc3b (155.1212, -6.6583, 24.83) link	3.13 null 14 [‘usb000hc3b’] 4.7 mb null us 31	155km WSW of Panguna, Papua New Guinea	1.05 340 [‘us’] REVIEWED 1.37E12 null earthquake	[‘cap’, ‘dyfi’, ‘general-link’, ‘geoserve’, ‘nearby-cities’, ‘origin’, ‘p-wave-travel-times’, ‘phase-data’, ‘scitech-link’, ‘tectonic-summary’] 600 1370253974

4. Query the first 5 earthquakes with ‘mag’ greater than 5.0 and ‘status’ equal to ‘REVIEWED’

```
SELECT * FROM earthquakes WHERE mag > 5.0 AND status = 'REVIEWED' LIMIT 2 ALLOW FILTERING;
```

```
usb000h700 | green | 5.6 | b000h700 | (-82.6475, 9.3873, 11.26) | http://earthquake.usgs.gov/ea
 usb000gz6h |  null |   1 | b000gz6h |    (159.993, 52.41, 43.7) | http://earthquake.usgs.gov/e
```

5. Count the number of earthquakes with ‘mag’ greater than 5 (ALLOW FILTERING is inefficient and should be avoided where possible)

```
SELECT COUNT(*) FROM earthquakes WHERE mag > 5 ALLOW FILTERING;
```

count
99

6. Query earthquakes by 'place' using the secondary index on 'place' (secondary indexes are less efficient than primary key queries but more efficient than using ALLOW FILTERING)

```
CREATE INDEX IF NOT EXISTS place_index ON earthquakes (place);
SELECT * FROM earthquakes WHERE place = '6km ENE of Desert Hot Springs, California';
```

id	alert	cdi	code	coordinates	detail	dmin	felt	gap	ids	mag	magType	net	place	rms	sig	sources	status	tsunami	type	types	tz	updated	url
ci15352805		3.5	35328905	(-116.4387, 33.983, 7.2)	https://earthquake.usgs.gov/fdsnws/event/1/query?eventid=ci15352805&format=geojson	0.04491	16	116	[ci15352805]	1.96	ml	ci	6km ENE of Desert Hot Springs, California	0.09	19	[ci]	AUTOMATIC	0	earthquake	[general-link, geoserve, nearby-cities, origin, scitech-link]	-420	1370254362540	https://earthquake.usgs.gov/earthquakes/eventpage/ci15352805

1.3.2 Hard queries

1.3.3 Complex queries

1.4 Appendices

1.4.1 Docker container definition

This is the `docker-compose.yml` file that was used to generate the container on which all the code of this project was run. It is part of our Git repository.

```
services:
cassandra:
    image: cassandra:latest
    ports:
    - 7000:7000
    - 7001:7001
    - 7199:7199
    - 9042:9042
    - 9160:9160
    healthcheck:
    test: ["CMD-SHELL", "[ $$(nodetool statusgossip) = running ]"]
    interval: 30s
    timeout: 10s
    retries: 5
    volumes:
    - ./src:/root/src
```